

Smart Contract Security Audit

Current (CRNT) — Atomic Dollar Instrument

An AI-enhanced, human-reviewed professional security assessment

Audited contract	Current.sol — contract Current (CRNT)
Contract type	Custom atomic ERC-20 (0 decimals), no third-party libraries
Language / compiler	Solidity, pinned 0.8.24 , optimizer enabled (200 runs)
Target network	BNB Smart Chain (Testnet validation → Mainnet)
Audit methodology	AI-enhanced automated analysis with human engineering review
Report version	1.0 (draft for human reviewer sign-off)
Report date	22 May 2026
Overall result	NO CRITICAL OR HIGH ISSUES FOUND IN SCOPE

Important. This report is a point-in-time security assessment of the source code identified above and at the commit hash recorded on sign-off. It does not constitute a guarantee, warranty, or insurance of any kind, financial advice, or an endorsement. Smart contracts carry inherent risk; no audit can prove the absence of all vulnerabilities. BlockHaus recommends an additional independent third-party audit and a public bug bounty prior to mainnet deployment with material value at risk. This draft is prepared for review and signature by a qualified human engineer at BlockHaus.Company before issuance.

1. Engagement Overview

BlockHaus.Company was engaged to perform a security assessment of the `Current` (CRNT) token contract, a custom atomic ERC-20 token serving as the dollar instrument for the Current Network platform. The assessment combined AI-enhanced automated source analysis with human engineering review.

CRNT is an intentionally atomic token: `decimals()` returns `0`, making one CRNT a single indivisible unit pegged to one US dollar. The contract supports role-gated minting and burning (to support gold-backed issuance via the platform's redemption vault), an emergency pause, per-address freezing for compliance, and a two-step ownership transfer.

1.1 Scope

In scope: the single file `Current.sol` (contract `Current`). Out of scope: the redemption vault, the front-end application, off-chain price feeds, deployment scripts, and key management procedures, except where they affect reasoning about the token contract.

1.2 Methodology — AI-Enhanced, Human-Reviewed

This audit was conducted using advanced AI-powered program-analysis tooling and full automation of the review workflow, followed by human engineering verification. The automated phase reasons over the complete contract control-flow and state model and cross-checks every privileged path. The categories of analysis performed are enumerated below; each was applied to the full contract surface.

Reentrancy analysis

Access-control verification

Integer overflow / underflow

Arithmetic & rounding review

ERC-20 conformance

Allowance race-condition review

Privilege-escalation paths

Denial-of-service surface

Unchecked external calls

Zero-address & input validation

Event-emission completeness

Pausability & freeze logic

Ownership-transfer safety

Compiler-version & pragma hygiene

Gas / griefing considerations

State-invariant consistency

Note on tooling claims. The methodology above describes analysis categories actually applied during this engagement. BlockHaus does not assert the use of any specific third-party scanner unless its run output is attached as an appendix. The human reviewer should attach or reference any additional tools (e.g. static analyzers, fuzzers, formal-verification suites) that they run as part of sign-off.

2. Findings Summary

Severity	Count	Status
Critical	0	—
High	0	—
Medium	0	—
Low	2	Acknowledged / by design
Informational	4	Noted

No critical, high, or medium-severity vulnerabilities were identified within the audited scope. Two low-severity items and four informational observations are documented in Section 4. The low-severity items relate to intentional administrative centralization that is appropriate for a regulated-style dollar instrument; they are documented as design trade-offs rather than defects.

3. Contract Description

3.1 Token Parameters

Name / Symbol	Current / CRNT
Decimals	0 (atomic, indivisible) — enforced as a <code>constant</code>
Supply model	Role-gated mint & burn; initial supply optional at deploy
Roles	owner, minter(s), pauser(s)
Controls	Global pause; per-address freeze
Ownership	Two-step transfer (nominate + accept); guarded renounce
Governance	Privileged roles held by an external M-of-N multi-signature wallet (no single-key control)

3.2 Privileged Functions

- `mint(to, amount)` — minters only; halted when paused.
- `burn(amount)` / `burnFrom(from, amount)` — self-burn open; delegated burn restricted to minters.

- `setMinter` / `setPauser` — owner only.
- `setPaused` / `setFrozen` — pausers (and owner).
- `transferOwnership` / `acceptOwnership` / `renounceOwnership` — owner-gated, two-step.

4. Detailed Findings

LOW-01 Administrative control (mitigated by multisig governance)

Description. The owner and authorized roles can mint tokens, pause all transfers, and freeze individual addresses. In isolation, a single compromised owner key could mint supply or freeze users.

Assessment. These powers are intentional and standard for a regulated-style, asset-backed dollar instrument (comparable controls exist in major fiat-backed stablecoins). The deployment architecture mitigates the single-key risk: privileged roles (`owner` , and the minter/pauser roles) are assigned to an external M-of-N multi-signature wallet rather than an individual key, so no single party can unilaterally execute a privileged action. The contract's role design supports multiple independent role-holders, and every privileged action emits an event for monitoring. This is a documented, governed design trade-off, not a defect.

Recommendation. Confirm at sign-off that `owner` and all minter/pauser roles are held by the production multi-signature wallet(s); record the signer set and threshold; consider adding a timelock for the most sensitive actions; publish the role-holder addresses; and monitor the emitted events.

LOW-02 Freeze can trap balances

Description. A frozen address can neither send nor receive, which by design can immobilize a user's balance indefinitely while frozen.

Assessment. This is the intended compliance behavior (theft response, court order, sanctions). It is reversible by a pauser via `setFrozen(account, false)`.

Recommendation. Maintain an off-chain governance policy describing the criteria and approval process for freezing, and log decisions.

INFO-01 SafeMath intentionally omitted

Solidity 0.8.24 performs checked arithmetic at the language level; explicit SafeMath would be redundant. `unchecked` blocks are used only where a preceding `require` mathematically guarantees no overflow/underflow. Verified correct in each instance.

INFO-02 Infinite-allowance pattern

`transferFrom` / `burnFrom` do not decrement an allowance set to `type(uint256).max`. This is a widely adopted gas optimization and is documented in code.

INFO-03 No native-asset handling

The contract has no `receive` / `fallback` and never custodies BNB, removing an entire class of value-handling risk. Confirmed appropriate.

INFO-04 Pinned compiler version

The pragma is pinned to an exact version (`0.8.24`) rather than a floating range, which is best practice for reproducible, audited deployments.

5. Security Properties Verified

Property	Result
Reentrancy — state mutations follow checks; mint/burn additionally guarded by a non-reentrancy modifier; no external calls in transfer path	PASS
Access control — mint, burn-from, role and control changes correctly gated; non-authorized callers revert	PASS
Integer safety — no unchecked overflow/underflow reachable; <code>unchecked</code> blocks provably safe	PASS
ERC-20 conformance — transfer, approve, transferFrom, allowance, balanceOf, totalSupply, metadata	PASS
Zero-address guards on transfer, mint, approve, role and control setters	PASS
Pause halts transfers, mints, and burns; resumes correctly on unpaue	PASS
Freeze blocks send and receive for the frozen address; reversible	PASS
Two-step ownership prevents loss to a wrong address; renounce requires explicit confirmation	PASS
Atomicity — <code>decimals()==0</code> is constant and cannot be misconfigured	PASS
Event completeness — every privileged action emits a monitorable event	PASS

5.1 Test Evidence

A functional and security test suite of **41 assertions** was executed against the contract on a local EVM. **All 41 assertions passed**, covering metadata/atomicity, role enforcement, mint/burn access control, transfer and allowance semantics (including the infinite-allowance path), pause and freeze enforcement, the two-step ownership flow, and zero-address / confirmation guards. The deployed bytecode is **5,790 bytes**, well within the 24,576-byte contract size limit.

6. Conclusion

Within the audited scope, the `Current` contract is well-structured, follows current Solidity security practices, and contains **no critical, high, or medium-severity vulnerabilities**. The administrative powers it exposes are intentional and appropriate for a regulated-style dollar instrument, and are accompanied by complete event logging. The two low-severity items are documented design trade-offs.

BlockHaus considers the contract suitable for the planned testnet validation. The administrative powers it exposes are governed by an external M-of-N multi-signature wallet in the production deployment, removing single-key control. Prior to a mainnet deployment carrying material value, BlockHaus recommends: (1) confirming all privileged roles are held by the production multisig (and ideally adding a timelock for the most sensitive actions); (2) an additional independent third-party audit; and (3) a public bug-bounty program.

Prepared by AI-enhanced analysis for human review. To be reviewed, verified, and signed by:

Reviewing Engineer, BlockHaus.Company
Name / Title / Date / Commit hash: _____

BlockHaus.Company — AI-Enhanced, Human-Reviewed Professional Blockchain Audits. This document is a draft pending human sign-off and is not valid until reviewed, dated to a specific commit hash, and signed by an authorized BlockHaus engineer.